
Vitrina app

Product catalog with role-based access

Solution architecture · .NET technical interview exercise

Oscar Sanchez

Software Developer

User story

As a Vitrina user,
I want to sign in and access the product catalog according to my role,
so that I can browse products or manage the catalog if I'm an administrator.

Acceptance criteria

Given a user is on the login page
When they submit valid credentials
Then they are authenticated and taken to the product catalog.

Given a user enters invalid credentials
When they try to sign in
Then access is denied and an error message is displayed.

Given an authenticated user is viewing the catalog
When they search, filter, paginate, or select a product
Then they can browse matching products and view product details.

Given an administrator is signed in
When they create, update, or delete a product
Then the change is validated and saved.

Given a non-administrator attempts to change a product
When the request reaches the backend
Then the operation is denied.

Delivered

Frontend: Login, session handling, product catalog, search and filters, product details, and admin forms.

Backend: Authentication with role-based access, product search and retrieval, and product create, update, and delete operations.

Design alternatives

Backend

A. EF Core with repositories (chosen)

- Entities mapped with EF Core configurations
- Migrations keep the schema in code
- Repositories are easy to mock in unit tests

B. Data layer on stored procedures

- Repositories call procedures (ADO.NET or Dapper)
- Full control of SQL and execution plans
- Manual mapping, logic split between C# and SQL

Frontend

A. Nx workspace, page containers (chosen)

- Nx workspace with the vitrina app on Angular 21
- Lazy-loaded features, containers hold the state
- Little boilerplate for this scope

B. Angular CLI app with NgRx

- Plain Angular project, no Nx workspace
- Redux state: actions, reducers, effects, selectors
- Predictable state, but more boilerplate

Backend architecture

Vitrina.Api

HTTP controllers, JWT authentication, authorization, Swagger and global error handling

Vitrina.Business

Product business services and validation

Vitrina.Dto

Request, response, pagination and validation data contracts

Vitrina.Data

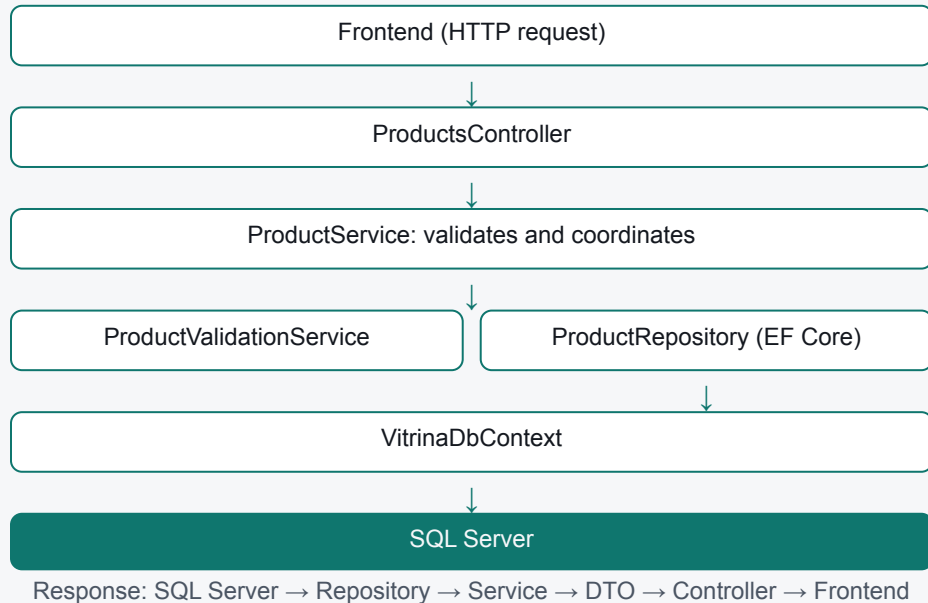
Entities, repositories, EF Core configuration, database context, migrations and seed data

Tests

Separate test projects for API, Business and Data

Backend request flows

Product request



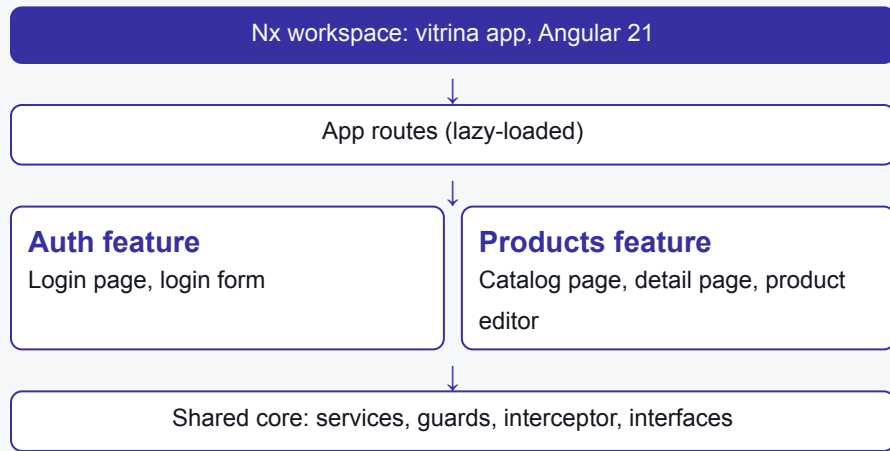
Login and authorization

1. Login request → AuthController → AuthService → UserRepository → SQL Server
2. Valid credentials return a JWT and session details to the frontend
3. Protected request → JWT validation → role authorization → controller action

Roles

- Admin and User: search and view products
- Admin only: create, update and delete products
- Program.cs configures services, middleware, authentication and EF Core migrations

Frontend architecture: Angular and Nx

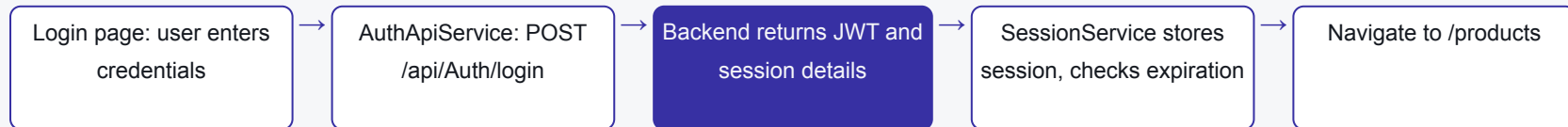


How UI parts connect

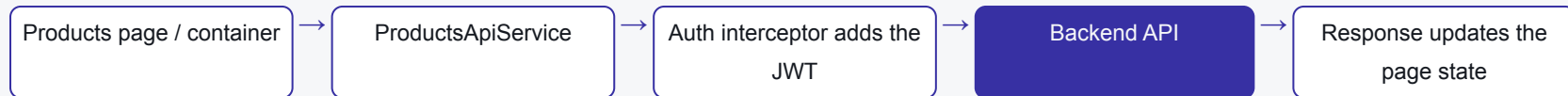
- Page containers coordinate state, forms, navigation, loading and API calls
- UI components display forms, lists and details through inputs and outputs
- Feature areas group routes, pages, components and interfaces for authentication and products
- Core provides shared application services and HTTP and security plumbing

Frontend flows: login and product API

Login and session



Protected product request



Guards and permissions

- authGuard requires a valid session for product routes; adminGuard restricts creation and editing routes
- The API independently validates the JWT and enforces administrator permissions
- Product screens support search, status filtering, pagination, details and administrator actions

Prompt for the GenAI tool

General API constraints

- Generate a RESTful task management API using the existing Vitrina solution, C# and ASP.NET Core Web API.
- Follow current layered architecture and conventions; do not create new projects or replace existing architecture.

Core objects and CRUD

Implement CRUD operations for tasks (ID, title, description, status, due date, associated user ID).
Assume existing User model and authentication are available.

Layered project implementation

- 1 **Api:** Add the tasks controller. Focused on HTTP, existing authentication, authorization, DI, and error patterns.
- 2 **Business:** Add the task service and business validation. Enforce users only access their own tasks.
- 3 **.Dto:** Add request and response DTOs. Do not expose database entities.
- 4 **Data:** Add task entity, EF Core configuration, DbSet, repository operations, migration. Follow existing VitrinaDbContext.

Common requirements and security

- **Authentication:** Required for all task endpoints (User ID from claims).
- **Access:** Users view/modify only own tasks.
- **Validation:** Validate required fields, input lengths, valid task status values.
- **Dates:** Use ISO 8601 dates.
- **Error Handling:** Appropriate HTTP codes, consistent error responses.
- **Registration:** Use existing DI methods.
- **Documentation:** Include OpenAPI and tests.

Before generating code: State any assumptions about task statuses and due-date handling.

Representative Sample: Controller and Service

```
[ApiController]
[Route("api/tasks")]
[Authorize]
public class TasksController(ITaskService taskService) : ControllerBase
{
    [HttpPost]
    public async Task<ActionResult<TaskDto>> Create(
        CreateTaskRequest request,
        CancellationToken cancellationToken)
    {
        var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

        if (string.IsNullOrEmpty(userId))
            return Unauthorized();

        var task = await taskService.CreateAsync(
            userId, request, cancellationToken);

        return CreatedAtAction(
            nameof(GetById),
            new { id = task.Id },
            task);
    }

    [HttpGet("{id:int}")]
    public async Task<ActionResult<TaskDto>> GetById(
        int id,
        CancellationToken cancellationToken)
    {
        var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

        if (string.IsNullOrEmpty(userId))
            return Unauthorized();

        var task = await taskService.GetByIdAsync(
            id, userId, cancellationToken);

        return task is null ? NotFound() : Ok(task);
    }
}
```

```
public sealed class TaskService(ITaskRepository repository) : ITaskService
{
    public async Task<TaskDto> CreateAsync(
        string userId,
        CreateTaskRequest request,
        CancellationToken cancellationToken)
    {
        if (string.IsNullOrEmpty(request.Title))
            throw new ValidationException("A title is required.");

        var task = new TaskItem
        {
            Title = request.Title.Trim(),
            Description = request.Description,
            Status = request.Status,
            DueDate = request.DueDate,
            UserId = userId
        };

        await repository.AddAsync(task, cancellationToken);
        return TaskDto.From(task);
    }

    public async Task<TaskDto?> GetByIdAsync(
        int id,
        string userId,
        CancellationToken cancellationToken)
    {
        var task = await repository.GetByIdAndUserIdAsync(
            id, userId, cancellationToken);

        return task is null ? null : TaskDto.From(task);
    }
}
```

Validating the AI's output

To review the generated code

- Build the solution and review the endpoint documentation.
- Test CRUD operations and verify their HTTP status codes.
- Confirm tasks are always associated with the authenticated user.
- Verify users cannot view or modify another user's tasks.
- Test unauthenticated requests and invalid input.
- Check that errors are returned clearly and consistently.

Edge cases and improvements

- Architectural improvements
- Unsupported task status.
- Invalid or ambiguous due dates.
- Task not found.
- User attempting to access someone else's task.
- Unauthenticated requests.
- Ensure the user identity comes from the authentication token, not the request body.